农田地图与远程控制系统——用户手册

版本：3.0

更新：2026 年 4 月 6 日

1. 系统功能概览

本系统是一个集地图绘制、路径规划、视频监控、远程遥控、状态反馈于一体的农业自动化车辆上位机控制平台。其核心目标是：

可视化作业环境：通过加载农田地理信息，构建农业无人工作场景。

循迹路线生成：自动计算最优拾取路径，减少无效行驶，提高效率。

远程监控观看：双路视频流推流与车辆状态灯指示，实现远程监看。

远程遥控界面：支持游戏手柄进行远程遥控实现紧急干预或手动操控。

农田边界下发：将农田边界发送至下位机用于绘制电子围栏，防止车辆越界。

2. 软件安装与启动

2.1 运行环境要求

操作系统：Windows 10/11（64 位）

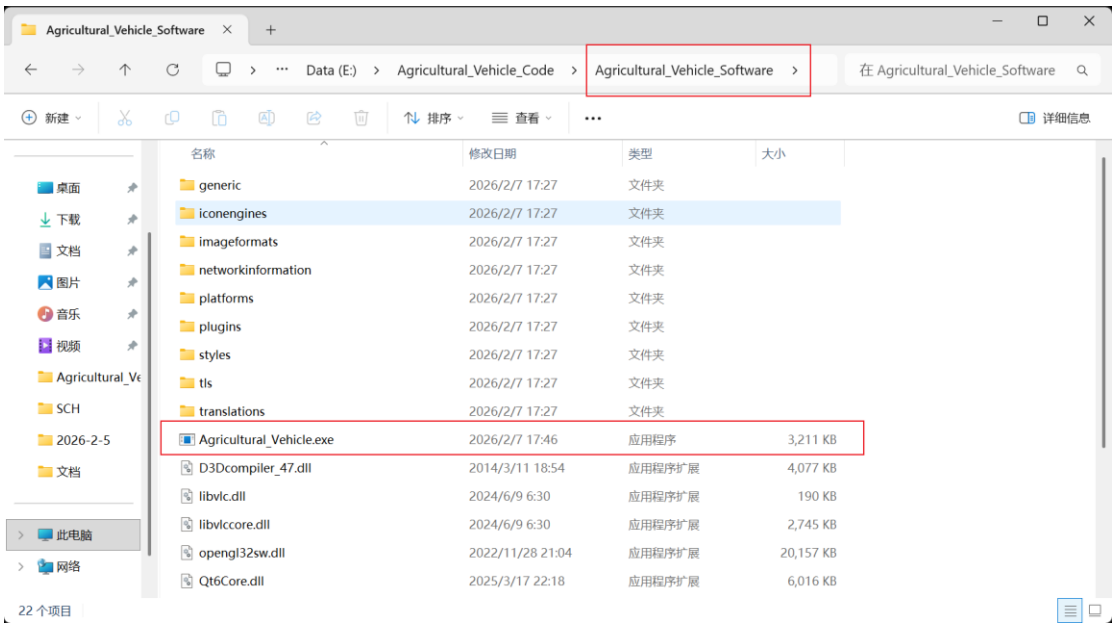
依赖库：

视频功能：需预装 VLC Media Player 3.x（仅 Windows 需要，Linux 可通过包管理器安装 libvlc-dev）（已准备好）

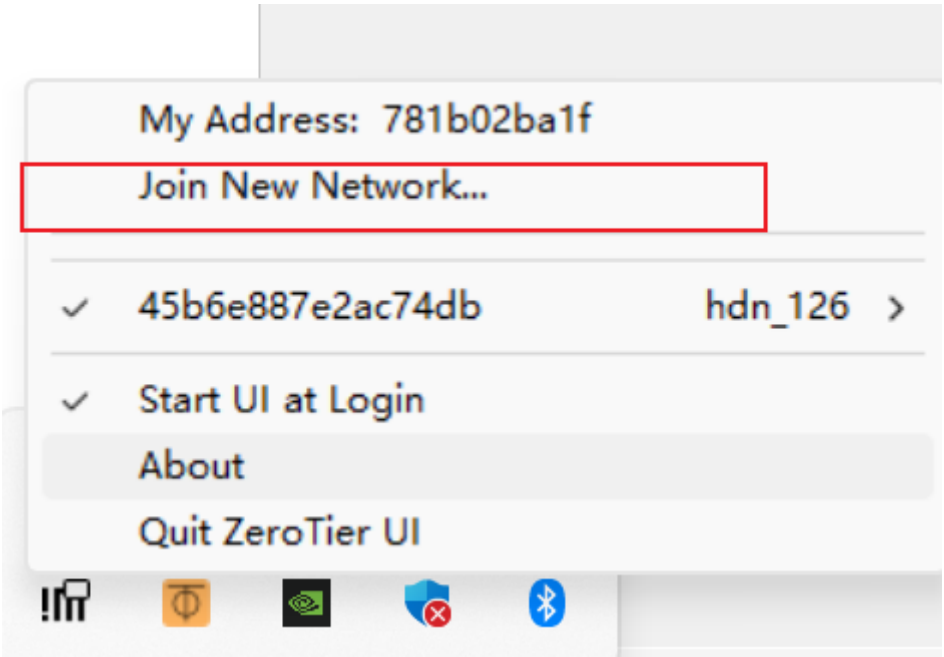
游戏手柄支持：标准 XInput 或 DirectInput 兼容手柄（如 Xbox 手柄）

2.2 启动步骤

将打包的软件目录（Agricultural_Vehicle_Software）复制到目标机器，目录中包含可执行文件 Agricultural_Vehicle.exe 以及程序运行所需要的 QT 依赖库、VLC 视频播放依赖库文件与手柄遥控 SDL3 依赖库文件。



确保网络连通（目标机器需要上网）：上位机与下位机（IP：10.244.241.213）在同一局域网。需要先在目标机器安装上 zerotier，官网下载：https://www.zerotier.com/，下载完后，需要将目标机器添加至 zerotier 网络做，具体操作为



添加后需要登录 zerotier 进行授权，目标机器和 jetson 设备才能进行通信。

随后双击运行程序。

主窗口将自动最大化，显示三个标签页：路径规划、监控、遥控。

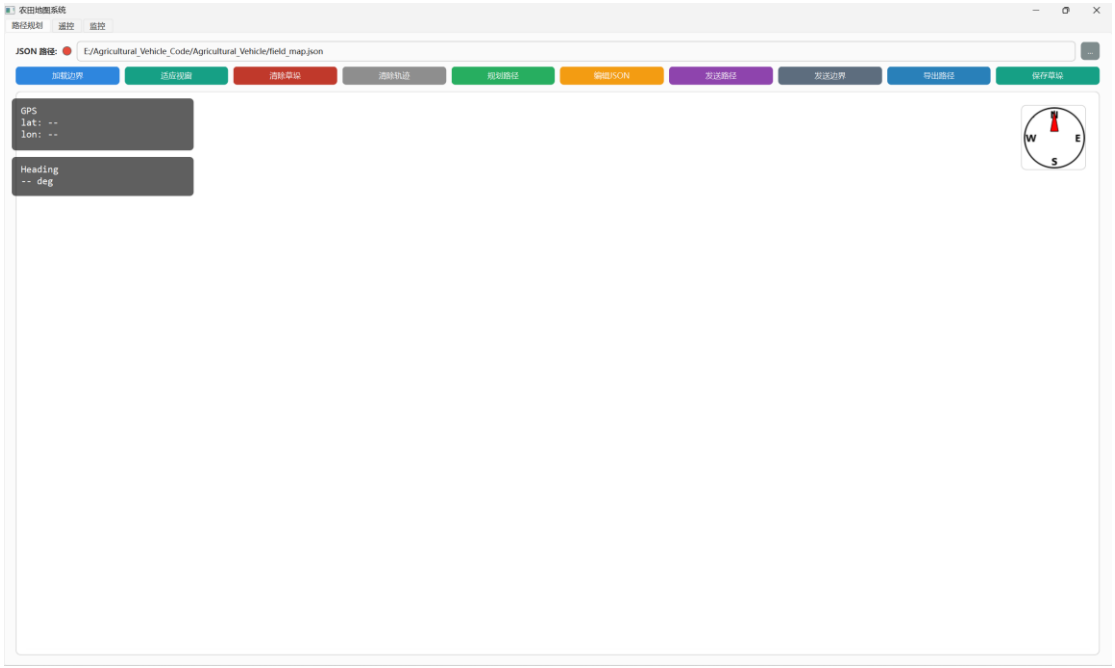


图 1 上位机——路径规划标签页

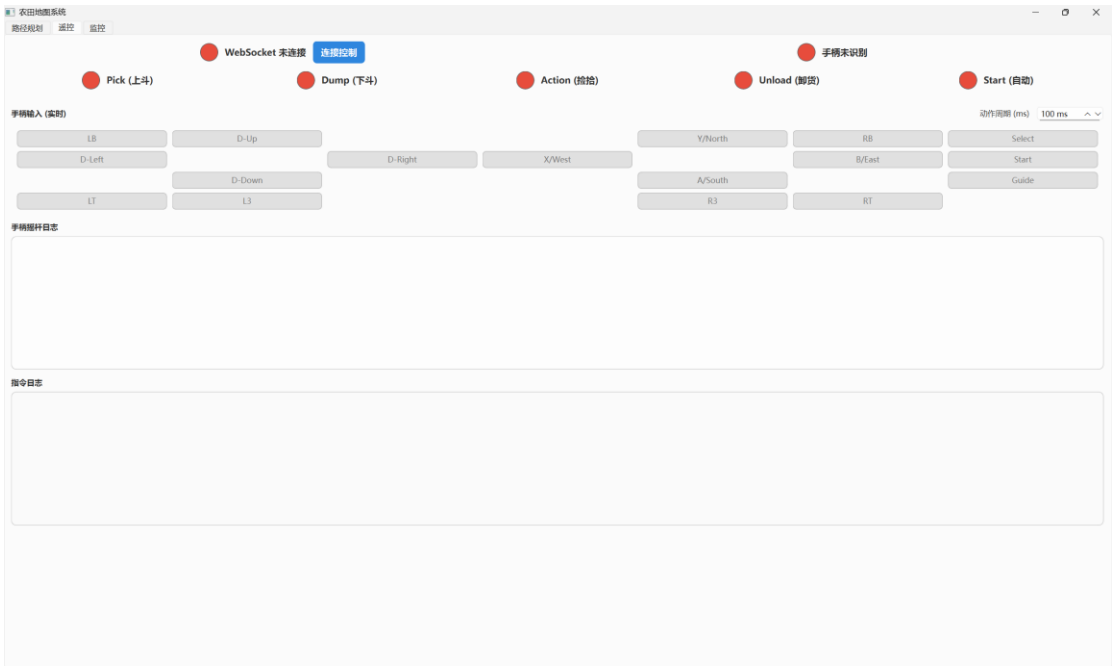


图 2 上位机——遥控标签页

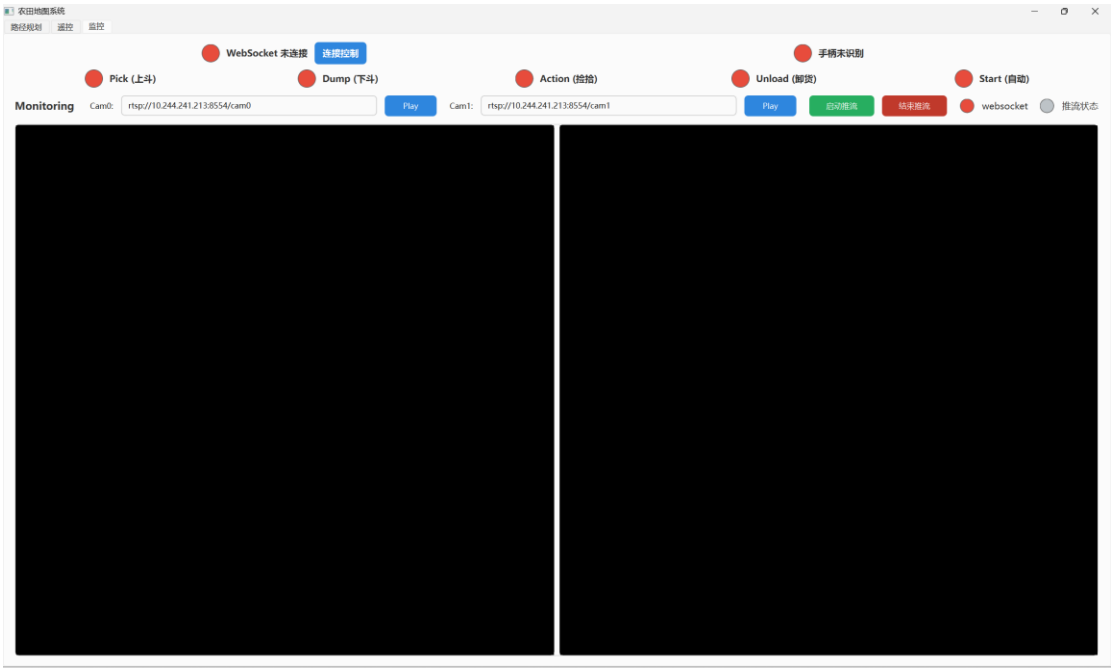


图 3 上位机——监控标签页

如果界面显示不全，将缩放比例调小一些，就可以显示完全。

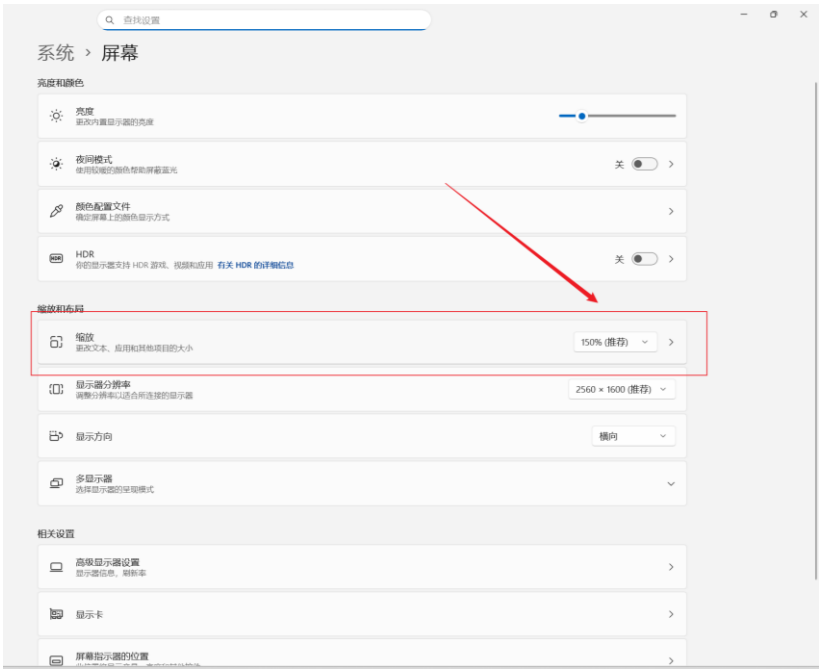


图 4 缩放比例调整

3. 功能模块详解

3.1 路径规划（Path Planning）

3.1.1 地图数据格式说明

系统使用 JSON 格式描述农田环境，文件结构如下（单位：经纬度为十进制度，角度为度）：

```
{
```

```
    "bales": [
        {
            "angle_deg": 22,
            "lat": 29.169589,
            "lon": 119.88300887
        }
    ],
    "end_point": {
        "angle_deg": -74,
        "lat": 29.16960001,
        "lon": 119.88287534
    },
    "field_boundary": [
        {
            "lat": 29.16958538,
            "lon": 119.88277034
        },
        {
            "lat": 29.16973144,
            "lon": 119.88281995
        },
        {
            "lat": 29.16966181,
            "lon": 119.88318839
        },
        {
            "lat": 29.16951946,
            "lon": 119.88312539
        }
    ],
    "obstacles": [
    ]
}
```


注意：field_boundary 必须是闭合多边形（首尾点不要求重

复），bales 和 obstacles 可为空数组。

3.1.2 操作流程

加载地图

在 “JSON 路径” 输入框中指定 .json 文件路径。

点击加载边界。成功后，图形视图将渲染：绿色实线：农田边界；黄色圆圈：草垛（中心点）；红色三角形：障碍物；蓝色菱形：车辆标识。图 4 为加载的农田边界。

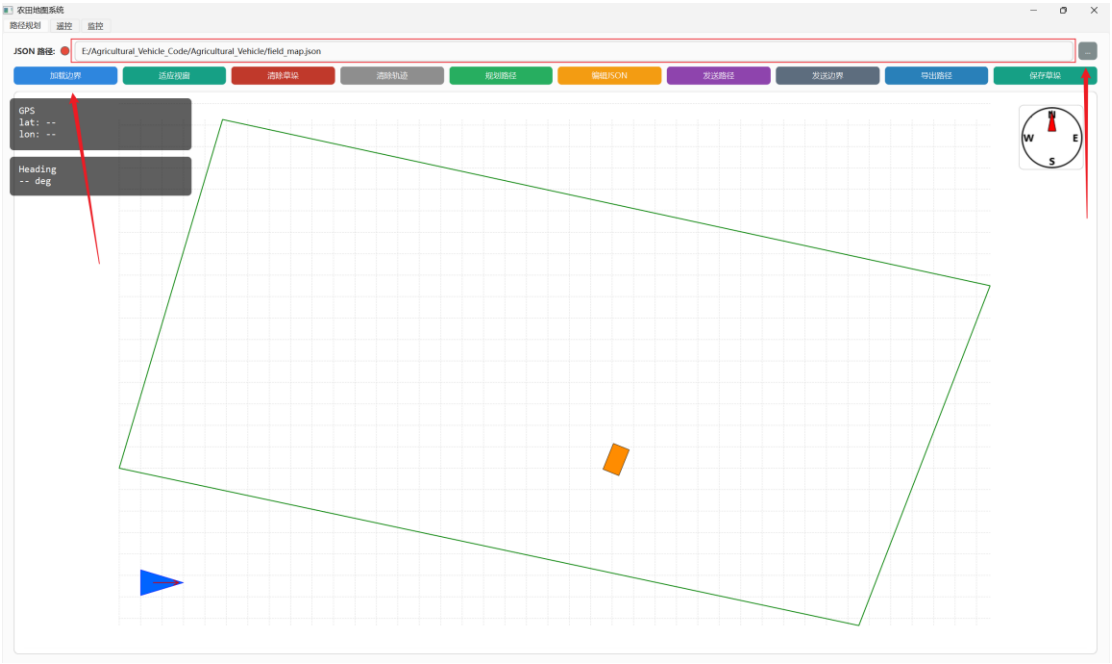


图 4 农田边界渲染图

如果文件内容不对或者路径不对，都不会加载成功，需要自查文件内容是否为 json 格式和路径是否正确。

编辑地图

点击编辑 JSON 按钮，弹出结构化编辑器。编辑器包含四个独立表格，分别对应 end_point（单行）、field_boundary、bales、obstacles。支持：

添加行：点击 “+” 按钮，在末尾新增一行，输入经纬度。

删除行：选中行后点击 “-” 按钮。

修改值：直接在单元格内编辑。

注意：草垛的方向只支持-180° ~180° 区间，其中，正北为 0° ，正东为 90° ，正西为-90° ，方向为草垛长边的朝向，由于草垛是对称的，如果草垛的朝向测出来为 250° ，则输入时，可以将 250° -180° =70° ，输入草垛的方向 70° 即可；终点的方向也是需要-180° ~180° 区间。

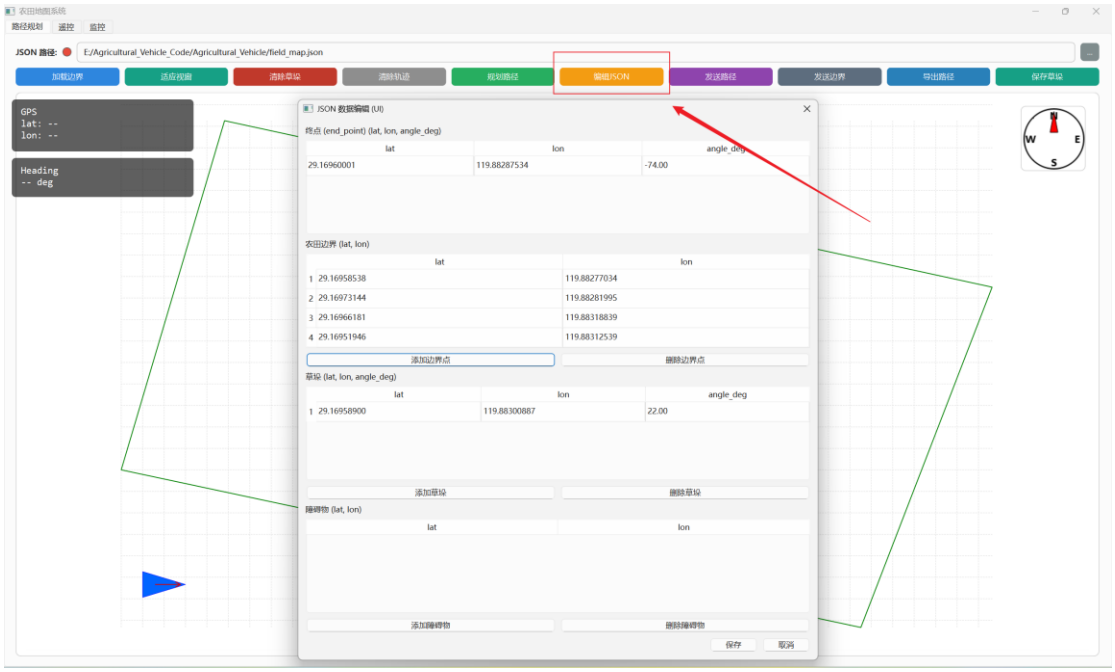
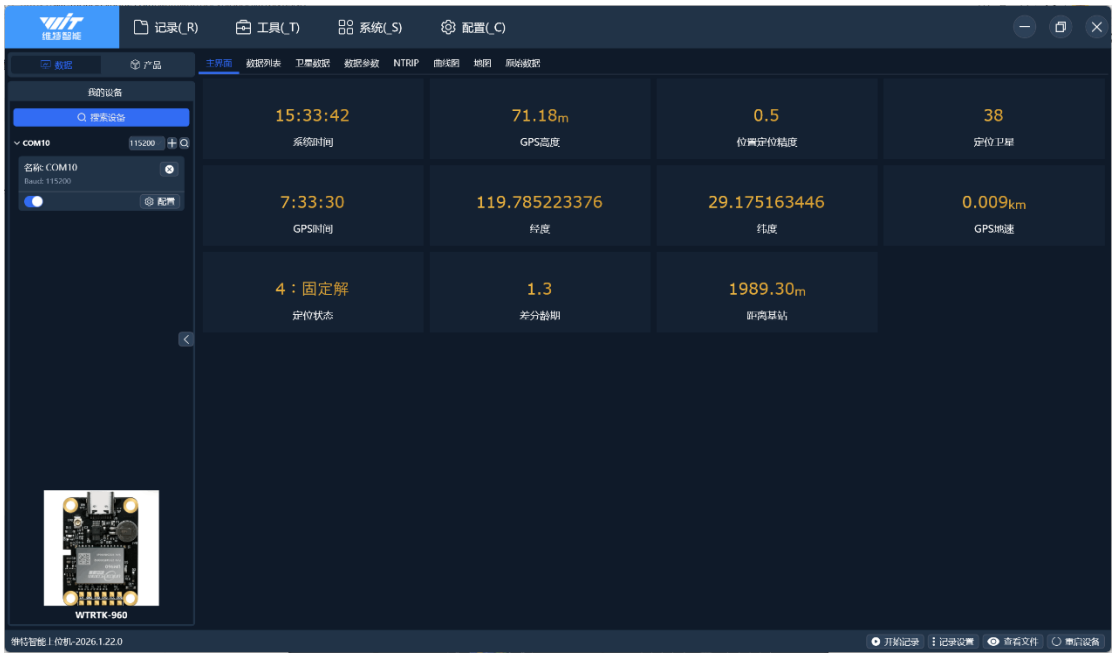


图 5 编辑 json 文件视图

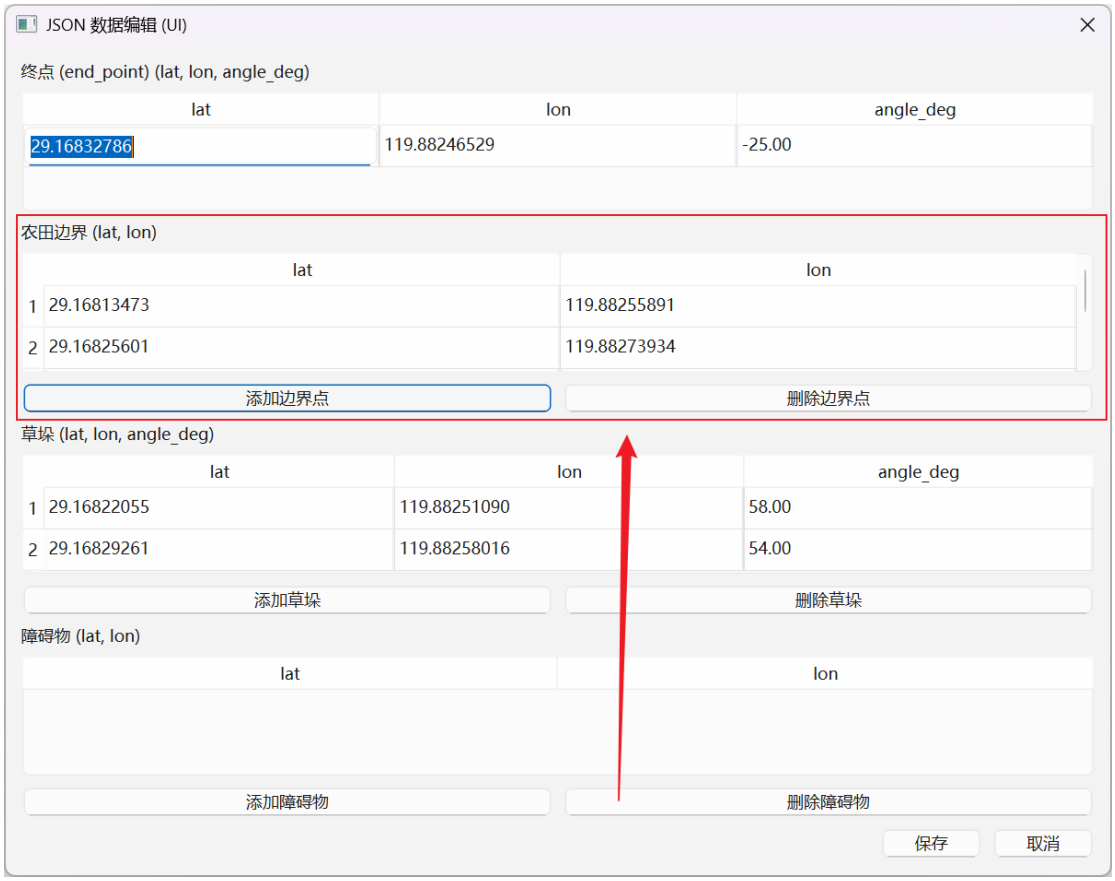
点击保存后，会覆盖原 JSON 文件，需要重启程序重新加载。

现在编辑地图采用上位机输入农田边界的经纬度，草垛的经纬度和方向来编辑 JSON 文件，来进行加载地图。

首先采用手持的 WTRTK-960 RTK 模块进行采集农田边界的经纬度：



依次采集每一个农田边界的经纬度，在以下界面中支持直接在原有的农田边界进行修改，也可以支持添加农田边界和删除农田边界：



草垛、终点、以及障碍物的添加和删除的流程与农田边界的添加和删除的方法一致。

RTK 记录



此区域用于配置 RTK 设备与软件的通信连接。

端口： 选择 RTK 设备连接的串口号（截图中默认为 COM8）。

刷新： 点击“刷新”按钮以重新扫描可用的串口列表。

波特率： 设置通信速率（截图中默认为 115200），需与 RTK 设备的波特率保持一致。

连接控制：

当未连接时，点击“连接”按钮建立连接。

当已连接时，按钮显示为“断开连接”，点击即可断开。

实时状态

此区域显示当前 RTK 设备的实时定位数据。

纬度 (Lat)： 显示当前的纬度坐标（例如：31.1912496）。

经度 (Lon)： 显示当前的经度坐标（例如：121.6280552）。

RTK 状态： 显示当前的定位解算状态（例如：单点解 (Single)）。

注意：若状态为“单点解”，定位精度较低；通常需等待达到“固定解 (Fixed)”状态以确保高精度记录。

记录操作

此区域用于记录不同类型的地理数据点。括号内的数字代表当前已记录的数量。

记录边界点 (0)： 点击以记录当前坐标为一个边界点。

记录障碍物 (0)： 点击以记录当前坐标为一个障碍物位置。

记录草垛 (0)： 点击以记录当前坐标为一个草垛位置。

记录起点： 点击以设定作业的起始点。

记录终点： 点击以设定作业的结束点。

数据导出

导出 JSON 文件： 点击此按钮，将当前记录的所有数据（边界点、障碍物、草垛、起终点等）保存为本地的 JSON 格式文件，以便后续处理或导入其他系统。

路径规划

前提：bales 数组非空，且存在 end_point。

点击规划路径。

算法逻辑：

起始点 = 地图中心（field_boundary 的几何中心）

使用最近邻（Nearest Neighbor）策略遍历所有草垛最后连接至 end_point。

视图显示：蓝色虚线：路径轨迹；紫色小方块：路径关键点（Waypoint）；黄色大长方形：目标草垛；

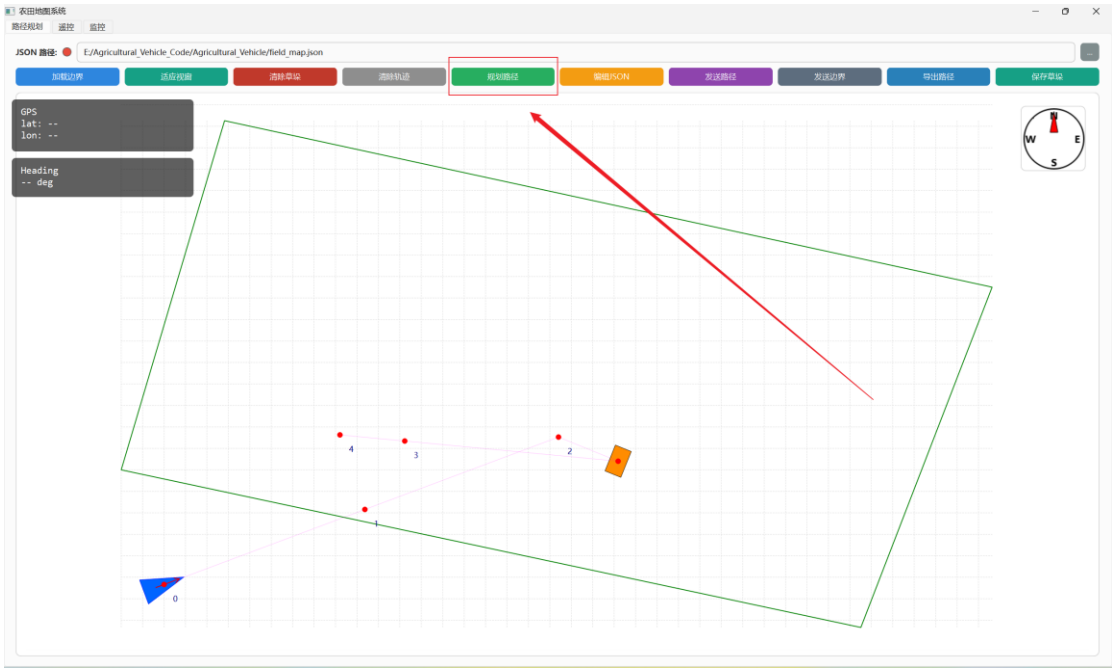


图 6 路径规划

下发数据

首先需要在 jetson 运行脚本：

```
huang@ubuntu:~/GPS$ ./start_remote_bridge.sh
Stopping old container (if any)...
remote_bridge
remote_bridge
Starting container with auto-install and run...
377171b7d70135d216d400244926bf2632cc99f943a5223ef8cb562aa86651cc
huang@ubuntu:~/GPS$ ls
gps_run.sh  ros1  server_gps_ws.py  server_heading_ws.py  start_remote_bridge_new.sh  start_remote_bridge.sh  teleop.sh  websocket_can_bridge.py  websocket_teleop.py
huang@ubuntu:~/GPS$
```

start_remote_bridge.sh 脚本的作用是将 jetson 和上位机的 websocket 通路连通，分别在 ROS2 的容器里启动上发经纬度给上位机以及上发方向给上位机。

```
huang@ubuntu:~/GPS_Path$ python3 receive_csv_all.py
WebSocket CSV server listening on 0.0.0.0:9002, saving to /home/huang/GPS_Path
Use /points for path points (6+ cols), /fence for boundary (idx,lat,lon)
```

然后在上位机中分别点击发送路径和发送边界，即向 jetson 发送 points.csv 和 fence.csv 文件，里面分别保存着路径规划和农田边界的信息。

```
huang@ubuntu:~/GPS_Path$ ls -al
total 32
drwxr-xr-x  2 huang huang 4096 Mar 25 17:46 .
drwxr-xr-x 42 huang huang 4096 Mar 19 18:17 ..
-rw-rw-r--  1 huang huang  120 Mar 25 17:46 fence.csv
-rw-rw-r--  1 huang huang  307 Mar 25 17:46 points.csv
-rwxrwxr-x  1 huang huang 2580 Feb  7 16:28 receive_csv_all.py
-rw-r--r--  1 huang huang 1657 Feb  5 11:54 server_csv_data.py
-rwxr-xr-x  1 huang huang 1975 Feb  4 16:23 server_csv_new.py
-rwxrwxrwx  1 huang huang 1949 Dec 23 12:02 server_csv.py
huang@ubuntu:~/GPS_Path$
```

发送路径：将路径点（不含草垛中心，仅轨迹点，带序号）按 CSV 格式通过 WebSocket (ws://10.244.241.213:9002/points) 发

送。

发送围栏：将农田边界通过 WebSocket(ws://10.244.241.213:9002/fence) 发送。

发送成功后，状态栏提示 “已发送”。

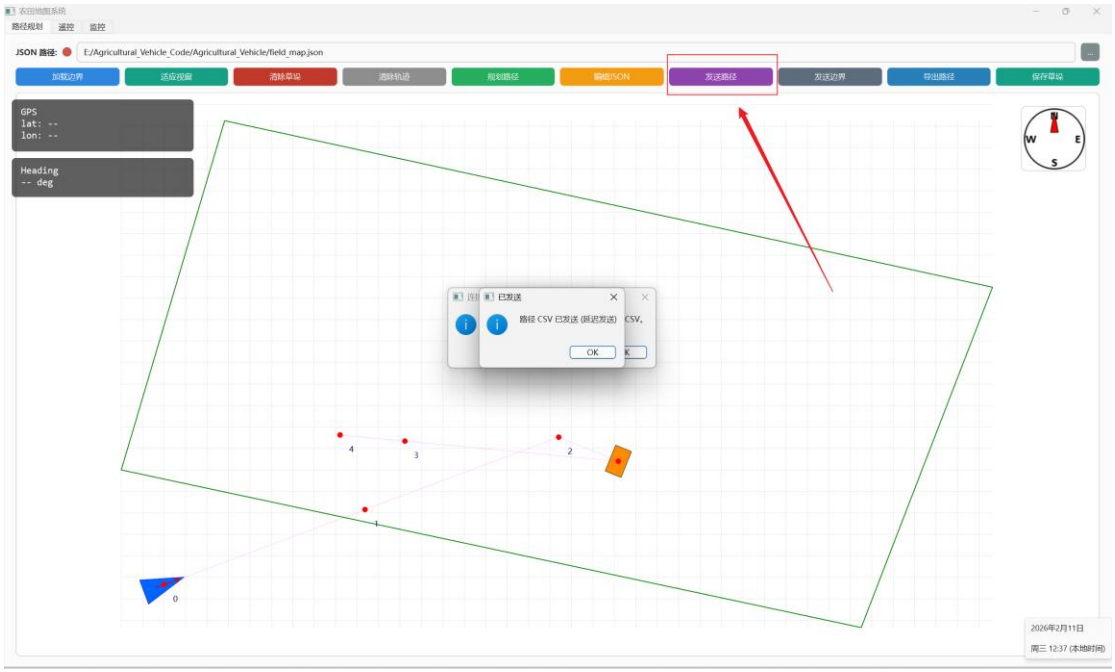


图 7 路径发送

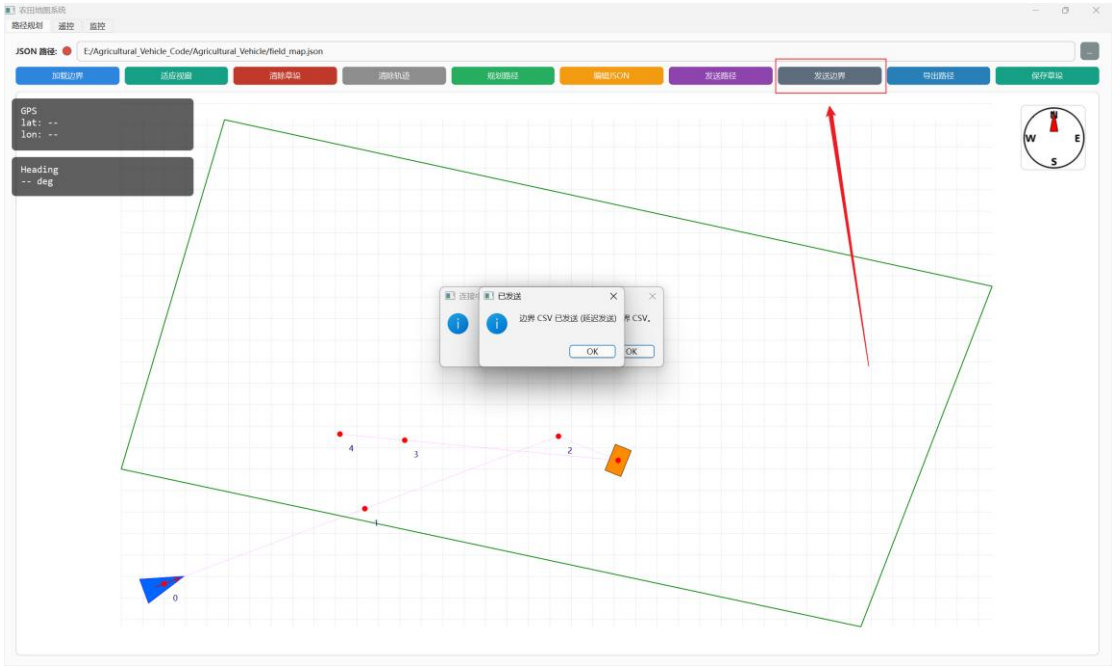


图 8 边界发送

随后可以在 jetson 的目录下看到 points.csv 以及 fence.csv，其中 points.csv 的内容为路径，fence.csv 的内容为农田边界。

导出路径

可以将路径规划的结果导出到 windows 下进行查看，做一个备份处理，默认保存路径为上位机的目录，默认文件名为：points.csv

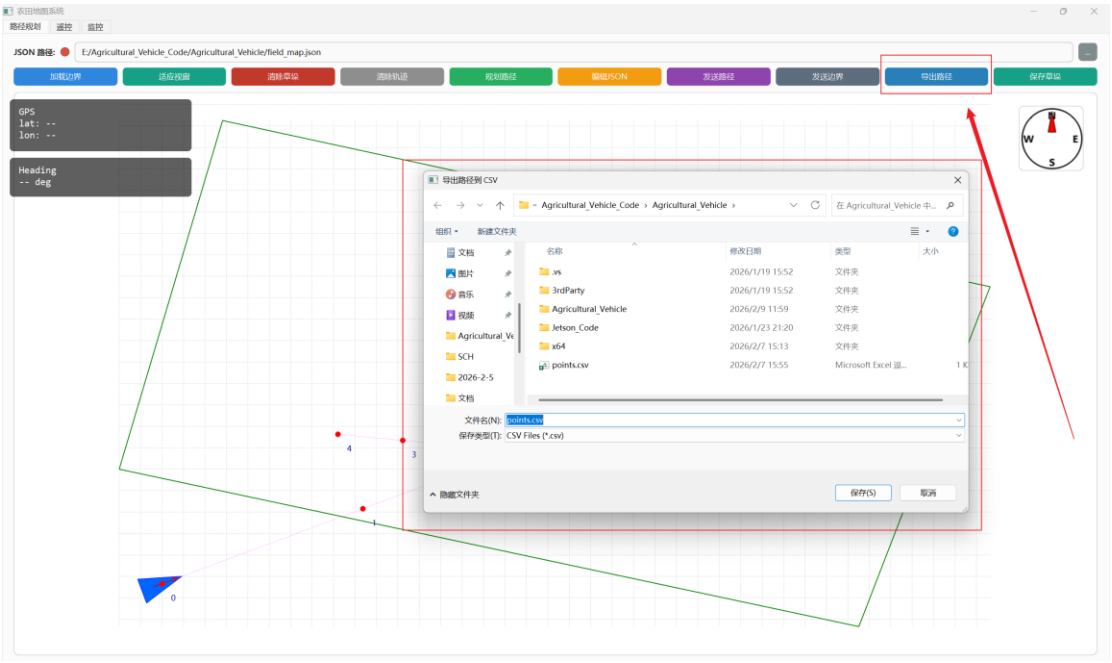


图 9 导出路径

3.1.3 视图交互

缩放：滚轮向前放大，向后缩小。

平移：左键拖拽。

旋转：右键水平拖拽（用于对齐作业方向）。

复位：点击 适应窗口，恢复初始视角。

3.2 监控（Monitoring）

3.2.1 视频流配置

由于视频流不能一直开着，所以建议手动开启脚本和关闭脚本：

```
huang@ubuntu:~$ cd monitor_usb/
huang@ubuntu:~/monitor_usb$ ls
config.sh  logs  mediamtx.yml  monitor.log  nohup.out  start_monitor_new.sh  start_monitor.sh  stop_monitor.sh  ws_control.py
huang@ubuntu:~/monitor_usb$ ./start_monitor_new.sh
=== 启动远程监控系统 ===
[1] 启动 MediaMTX 容器...
afe9077c462858471433940b3a4188e7ad4d3aca09a5c3deba34258eb387cb65
[2] 启动 2 路 GStreamer 推流...
  -> cam0 采集: /dev/video1  推流: rtsp://127.0.0.1:8554/cam0
  -> cam1 采集: /dev/video3  推流: rtsp://127.0.0.1:8554/cam1
=== 全部启动完成 ===
播放地址:
  rtsp://127.0.0.1:8554/cam0
  rtsp://127.0.0.1:8554/cam1
日志目录: /home/huang/monitor_usb/logs
```

图 10 执行监控开启脚本

```
huang@ubuntu:~/monitor_usb$ ./stop_monitor.sh
=== 停止监控推流 ===
kill 79057
kill 79069
mediamtx
=== 已停止 ===
```

图 11 执行监控关闭脚本

默认地址：

前摄像头：rtsp://10.244.241.213:8554/cam0

后摄像头：rtsp://10.244.241.213:8554/cam1

点击 play 可以开启视频播放，推流状态可以指示推流是否正在进行，蓝色表示正常推流，灰色表示推流关闭。

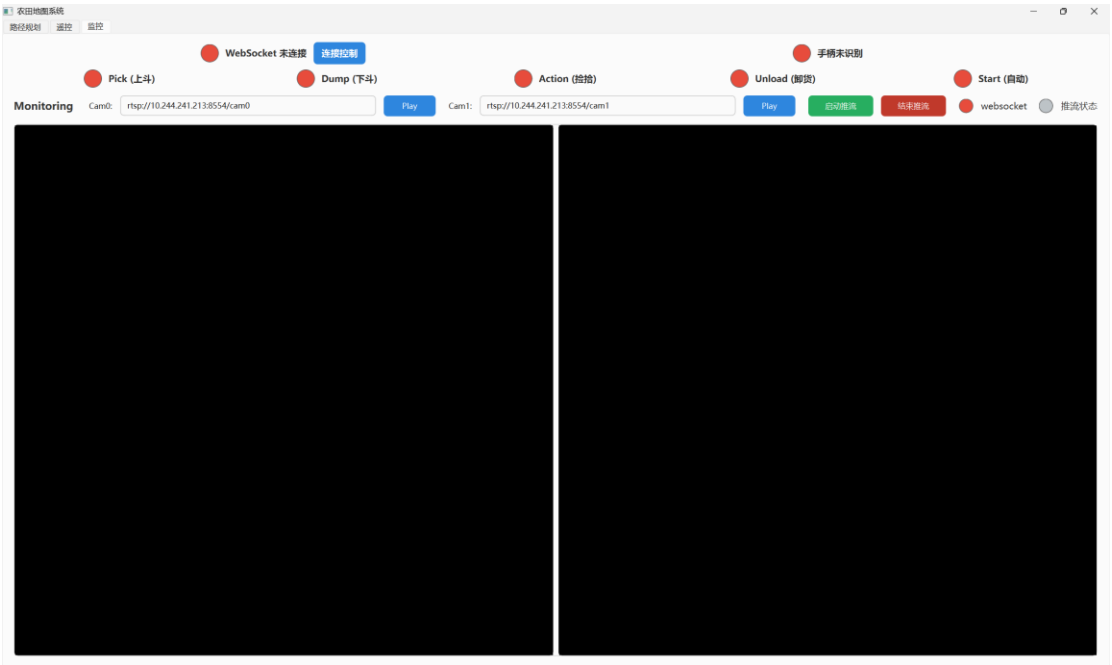


图 12 监控播放界面

可手动修改地址以适配不同设备。

由于 USB 摄像头插拔 jetson 后，会导致设备名改变，如果遇到推流不成功，需要查看 USB 摄像头识别成的设备名来进行更改。

```
huang@ubuntu:~/GPS_Path$ v4l2-ctl --list-devices
NVIDIA Tegra Video Input Device (platform:tegra-camrtc-ca):
        /dev/media0

vi-output, imx219 9-0010 (platform:tegra-capture-vi:1):
        /dev/video0

USB 2.0 Camera: USB Camera (usb-3610000.xhci-2.2.2):
        /dev/video1
        /dev/video2
        /dev/media1

USB 2.0 Camera: USB Camera (usb-3610000.xhci-2.2.3):
        /dev/video3
        /dev/video4
        /dev/media2

huang@ubuntu:~/GPS_Path$
```

通过 v4l2-ctl --list-devices 会列出所有视频设备及其对应的

/dev/video*节点，方便区分 USB 摄像头与 CSI 摄像头。

```
huang@ubuntu:~/monitor_usb$ cat config.sh
#!/bin/bash

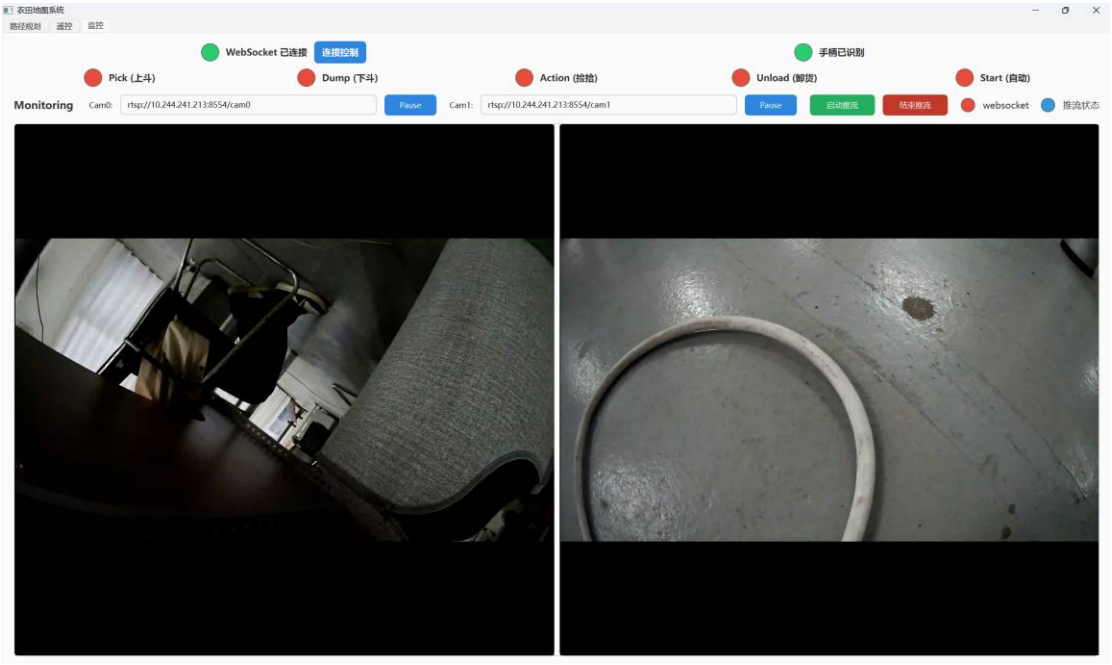
RTSP_SERVER="127.0.0.1"
RTSP_PORT=8554

CAM_NAMES=("cam0" "cam1")
CAM_DEVS=(
 # "/dev/v4l/by-path/platform-3610000.xhci-usb-0:2.1.4:1.0-video-index0"
 # "/dev/v4l/by-path/platform-3610000.xhci-usb-0:2.1.2:1.0-video-index0"
 "/dev/video1"
 "/dev/video3"
)

CAM_WIDTH=1920
CAM_HEIGHT=1080
CAM_FPS=30

CAM_BITRATE_KBPS=2000
```

如果发生更改，需要更改 config.sh 下面的摄像头节点映射。



3.2.2 控制逻辑

Play/Pause：控制本地 VLC 播放器的启停，不影响下位机推流。

启动推流：向 ws://10.244.241.213:9004 发送 {"cmd":"start"}，通知下位机开始编码 RTSP 流。

结束推流：发送 {"cmd":"stop"}，并停止本地播放。

3.2.3 状态指示区（顶部状态栏）

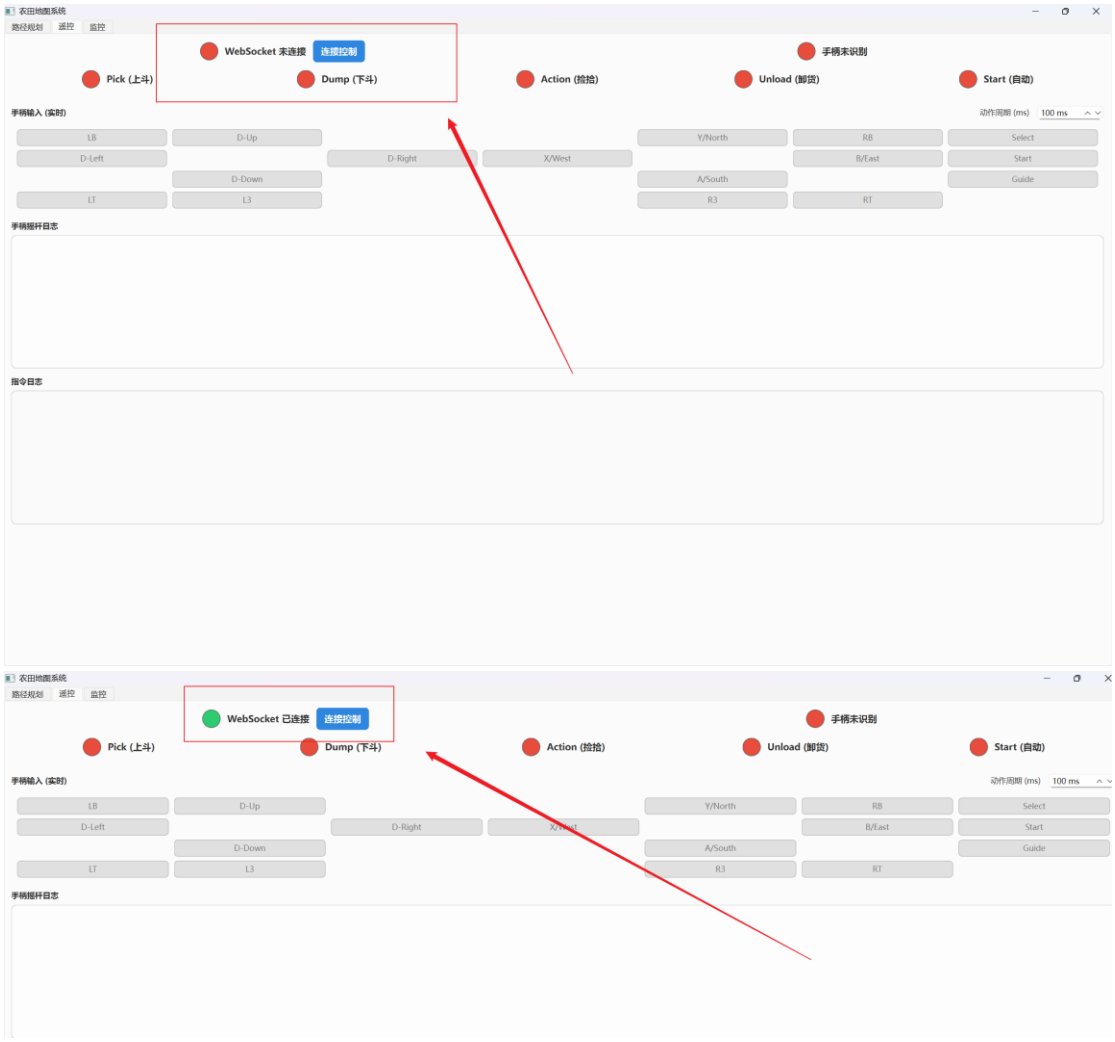
指示项	颜色/状态	含义说明
WebSocket	绿色/红色	是否连接遥控 WebSocket
手柄识别	“已连接”/“未连接”	SDL3 是否检测到手柄
Pick	绿色亮起	上斗动作激活
Dump	绿色亮起	下斗动作激活
Action	绿色亮起	抓取机构动作激活

Unload	绿色亮起	卸货动作激活
Start Mode	“自动”/“遥控”	当前车辆运行模式

3.3 遥控（Remote Control）

3.3.1 连接流程

在使用前一定需要点击连接控制按钮，等到连接指示灯为绿色的时候才能正常通信。



手柄未连接时，手柄识别指示灯为红色，手柄连接时，手柄指示灯为绿色。







手柄的模式一定要进入模拟模式,红色指示灯亮代表模拟模式,红色指示灯不亮为数字模式,可以通过 ANALOG 按钮来切换。

确保下位机已启动遥控服务（监听 :9010）。点击连接控制。成功后，按钮变为断开控制，状态灯变绿。手柄插上后，手柄指示灯也会变绿。

3.3.2 手柄映射表（Xbox 风格）

手柄按键	功能	发送的 JSON 命令示例
RT	上斗 (Pick)	{"key":"pick","value":1}
RB	下斗 (Dump)	{"key":"dump","value":1}
LT	抓取 (Action)	{"key":"action","value":1}
LB	卸货 (Unload)	{"key":"unload","value":1}
Start	切换自动/遥控模式	{"key":"teleop","value":0/1}
Select	自定义命令 X	{"key":"x"}
左摇杆 Y	前进/后退	{"key":"throttle","value":±0.0~1.0}

注意：摇杆值为归一化浮点数（-1.0 ~ +1.0），松开时自动回中（发送 0.0）。左边摇杆控制前进与后退，左边摇杆控制旋转。

3.3.3 日志查看

所有发出的遥控指令均记录在下方文本框。

日志格式：[时间] -> {JSON}，便于排查通信问题。

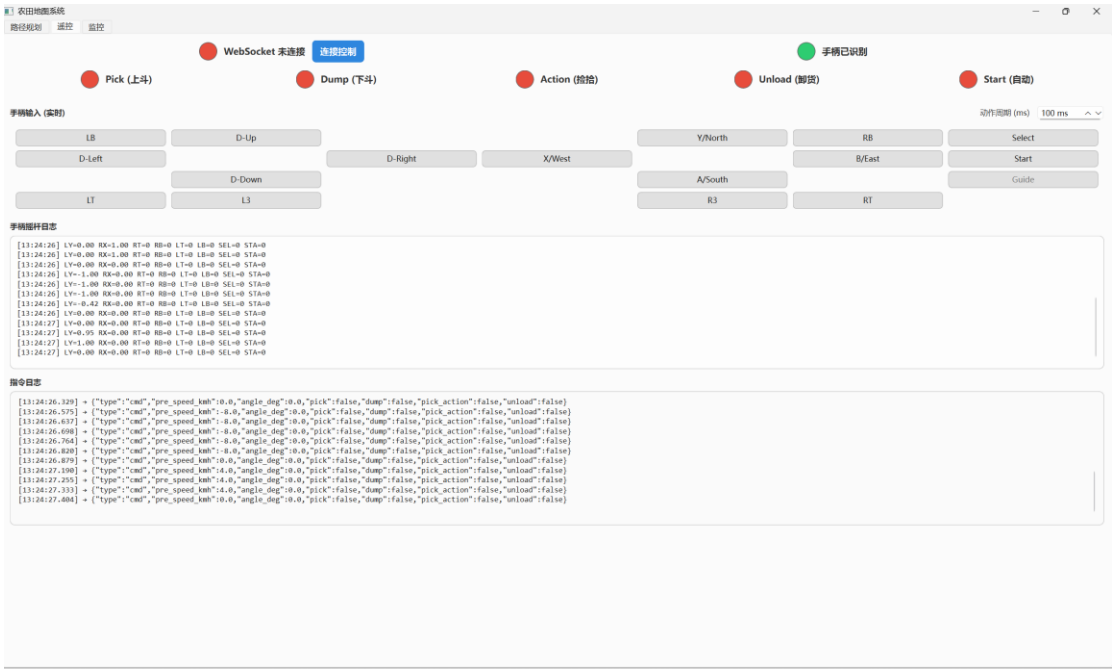


图 12 遥控界面